

Introduction: The Trap of "Just Counting"

If you are preparing for a software engineering interview, you have likely encountered the classic "Valid Parentheses" problem on LeetCode. At first glance, it seems deceptively simple. You have a string containing only opening and closing brackets—parentheses `()`, square brackets `[]`, and curly braces `{}`. Your job is to determine if every opening bracket has a corresponding closing bracket of the *same type* in the correct order.

The intuitive reaction for many developers is to reach for a counter. "I'll just count the openers and closers," they think. "If the counts match, it's valid." While this logic holds for simple cases like `()`, `[]`, or `{[()]}`, it catastrophically fails when faced with interleaved brackets like `( [ ] )`.

This is where interviewers love to test you. They aren't just checking if you know how to count; they are checking if you understand **order**, **matching types**, and **Last-In, First-Out (LIFO)** logic. This problem serves as a foundational lesson in why specific data structures exist. By mastering the stack approach here, you build an intuition that will help you solve complex parsing problems later on. In this guide, we will walk through the pitfalls of naive solutions, construct the optimal algorithm using Python's `stack` pattern, and analyze the performance gains.

Problem Overview

The core task is straightforward: given a string consisting only of characters `'('`, `')'`, `'['`, `']'`, `'{'`, and `'}'`, return `true` if the input string is valid, and `false` otherwise.

A string is considered valid only if two conditions are met simultaneously: 1. **Type Matching:** Every opening bracket must be closed by the *same type* of bracket. For example, an open parenthesis `(` must be closed by a parenthesis `)`, not a square bracket `]`. 2. **Order Preservation:** Brackets must close in the reverse order they were opened. If you push A then B onto the structure, B must be popped first.

The "naive" counter approach fails because it ignores these two constraints. It treats all open brackets as identical resources that can be closed by any available closing bracket at any time. The problem is designed to expose this flaw immediately.

Example Analysis

Let's break down why intuition often leads us astray with specific examples.

Example 1: Valid Input * **Input:** `"([)]"` * **Naive Expectation:** If we just count, we see 2 openers and 2 closers. We might incorrectly conclude this is valid. * **Actual Logic:** 1. We encounter `(` (open). Current state: `(` is waiting. 2. We encounter `[` (open). Current state: We have `(` waiting, then `[` sitting on top of it. 3. We encounter `)` (close). This tries to close the *last* opened item, which is `[`. But `)` does not match `[`. 4. **Result:** Invalid immediately.

Example 2: Valid Input * **Input:** `"()"` * **Logic:** Open `(`, then Close `)`. They match perfectly. Stack becomes empty at the end. * **Result:** Valid.

Example 3: Empty Input * **Input:** `""` * **Logic:** The stack starts empty and remains empty throughout. No unmatched brackets. * **Result:** Valid.

Intuition: Discovering the Stack Pattern

How does an experienced engineer arrive at the solution without immediately writing code? It starts with a mental model of **nesting**.

Think about HTML tags. If you write `<div>`, then inside it you write `<span>`, when you want to close them, you cannot close the `div` before the `span`. You must close the `span` first. The browser waits for the most recent open tag before closing the older ones.

This "Last In, First Out" behavior is the definition of a **Stack**.

In our bracket problem: 1. When we see an opening bracket, we are *adding* to our nesting depth. We must remember what we added so we can close it later. We **push** onto the stack. 2. When we see a closing bracket, we are trying to *resolve* the most recent open bracket. We look at the top of our stack. If it matches, great—we resolve it and remove it (**pop**). If it doesn't match or the stack is empty, we have an error in ordering or type.

The Key Insight: You do not need a complex lookup table if you store the *closing* character on the stack while pushing the *opening* one. * See `(` ? Push `)` (the expected partner). * See `[` ? Push `]`. * Now, when we see a closing bracket, we simply check if it equals the top of the stack.

Wait—why store the closing character? It simplifies the logic! Instead of asking "What opens this closing bracket?", we ask "Is this closing bracket what I was waiting for?" This avoids nested conditionals and makes the code cleaner.

Brute Force Solution: The Counter Trap

Let's formalize the naive approach that leads to failure on the test case `([)]`.

The Idea: Maintain three counters (or one counter tracking net balance) for each bracket type, or simply a single integer count assuming symmetry. Increment for every opener, decrement for every closer. If the total sum is zero at the end, assume valid.

Algorithm Steps: 1. Initialize `count = 0`. 2. Iterate through the string. 3. If char is `(`, `[`, or `{`: increment `count`. 4. If char is `)`, `]`, or `}`: decrement `count`. 5. If `count < 0` at any point (more closers than openers), return `False`. 6. At the end, if `count == 0`, return `True`.

Advantages: * Extremely simple to implement. * Runs in $O(n)$ time. * Uses $O(1)$ space (assuming single counter).

Disadvantages: * **Fundamentally Flawed Logic:** It ignores bracket *types*. `([)]` has two opens and two closes, so the counter ends at 0, but the types don't match. * To fix this properly with counters, you would need a complex state machine checking every combination, which defeats the purpose of a simple counter.

Complexity: * **Time:** $O(n)$ * **Space:** $O(1)$ (for the single integer counter).

Optimal Solution: The Stack Approach

The optimal solution leverages the data structure that matches the problem's topology perfectly: a **Stack**.

The Algorithm: We will use Python's built-in `list` as a stack. Lists in Python have an $O(1)$ `append/pop` at the end, making them efficient for this purpose.

Step-by-Step Logic: 1. Initialize an empty list called `stack`. This will act as our LIFO container. 2. Create a mapping (dictionary) that maps every *closing* bracket to its corresponding *opening* bracket. For example: `{ '): '(', ']': '[', '}': '{' }`. This allows us to quickly check "Was the expected opener this one?". 3. Iterate through each character in the input string. * **Case A (Closing Bracket):** If the current character is `)`, `]`, or `}`: * We need to match it against the most recent open bracket. * Check if the `stack` is empty. If it is, there is no matching opener yet. Return `False`. * Look at the top of the stack (`stack[-1]`). If this value matches our map (e.g., top is `(` and current is `)`), then we found a match. **Pop** the stack to remove the matched pair. * If they don't match, the order or type is wrong. Return `False`. * **Case B (Opening Bracket):** If the current character is `(`, `[`, or `{`: * Push it onto the `stack`. We are recording that we have an open bracket waiting for a specific partner. 4. After iterating through the entire string: * Return `True` only if the `stack` is **empty**. *

Why? If the stack is not empty, it means we encountered opening brackets that never found a closing partner (e.g., input "(").

Why this works: The stack enforces strict ordering. We can only close the bracket that was opened *last*. This naturally handles nested structures like `([])` correctly and fails fast on `([])` .

Python Code

Here is the production-quality solution in Python. Note the clean separation of logic and the use of a list as a stack.

```
def isValid(s: str) -> bool:
    """
    Determines if a string of brackets is valid using a stack approach.

    Args:
        s (str): A string containing only '(', ')', '{', '}', '[' and ']'.

    Returns:
        bool: True if the string is valid, False otherwise.
    """
    # We use a list to act as our stack.
    # O(1) operations for append() and pop().
    stack = []

    # Map closing brackets to their opening counterparts.
    # This acts as our lookup table for validation.
    bracket_map = {
        ')': '(',
        ']': '[',
        '}': '{'
    }

    # Iterate over every character in the string
    for char in s:
        # If char is a closing bracket, we must check against stack top
        if char in bracket_map:
            # Base case: Stack empty means no opener to match this closer
            if not stack:
                return False

            # Pop the top element. We need to compare it with the map.
            # stack[-1] is the most recent opening bracket pushed.
            top_element = stack.pop()

            # If the popped opener doesn't match the expected opener for this closer, fail
            if bracket_map[char] != top_element:
                return False
```

```

    # If char is an opening bracket (or simply not in the map dict)
    # Push it onto the stack to wait for a matching closer.
    else:
        stack.append(char)

# Final validation: The string is only valid if no openers remain on the stack.
# Any remaining items mean we had unclosed brackets like "(".
return len(stack) == 0

# --- Test Cases ---
print(isValid("()"))           # True
print(isValid("()[{}]")       # True
print(isValid("{}")           # False
print(isValid("([)]")         # False (The classic trap!)
print(isValid("{}[]")         # True
print(isValid(""))            # True

```

Code Walkthrough

Let's dissect the implementation line-by-line to understand the mechanics.

1. **stack = []** : We initialize an empty list. In Python, this is the standard way to implement a stack using a dynamic array.
2. **bracket_map** : We define a dictionary where keys are closing brackets and values are the matching opening brackets. This allows us to answer "Does this `)` match the top of the stack?" in constant time $O(1)$ without complex nested `if/else` chains.
3. **for char in s** : We traverse the input string character by character.
4. **if char in bracket_map** : This checks if the current character is a *closer*.
 - If it is a closer (e.g., `)`), we must verify that there is an unmatched opener waiting for it.
 - **Critical Check:** We first check `if not stack`. If the stack is empty, there is no opener to match the current closer (like encountering `)` at the start of a string). We immediately return `False`.
 - **The Pop Operation:** If the stack isn't empty, we grab the top item: `top_element = stack.pop()`. Note that `.pop()` removes the item, updating our state.
 - **The Comparison:** We check if `bracket_map[char]` (the expected opener) equals `top_element`. If they differ, e.g., stack has `[` but we have `)`, we return `False`.
5. **else** : If the character is an opener (like `(`), it doesn't exist as a key in `bracket_map`. We simply push it onto the stack: `stack.append(char)`. This records that we are "inside" a parenthesis.
6. **return len(stack) == 0** : After the loop finishes, the stack represents any unclosed brackets. If it's empty, every opener found its matching closer in the correct order.

Dry Run: Tracing ([)]

Let's manually walk through the algorithm with the tricky input ([)] .

Step	Character	Action Taken	Stack Content (Visual)	Notes
1	(	Open bracket detected. Push to stack.	[(]	Top is (
2	[	Open bracket detected. Push to stack.	[[(]	Top is [
3	)	Closer detected. Check stack top.	-	Stack top is [. Map says) expects (. [!= (.
4	Fail	Mismatch found! Return False .	-	Invalid immediately.

Contrast with Valid Input ([{}]) : 1. (-> Push. Stack: ['('] 2. [-> Push. Stack: ['(', '[', '[{'] 3. { -> Push. Stack: ['(', '[', '{'] 4. } -> Closer. Top is {. Match! Pop. Stack: ['(', '[', '['] 5.] -> Closer. Top is [. Match! Pop. Stack: ['(', '['] 6.) -> Closer. Top is (. Match! Pop. Stack: [] 7. **End.** Stack is empty. Return True .

Complexity Analysis

The performance of this solution is highly efficient, suitable for large-scale inputs.

- **Time Complexity: $O(n)$**
 - We iterate through the string exactly once.
 - Every operation inside the loop (checking dictionary membership, popping a list, pushing to a list) takes constant time $O(1)$.
 - Therefore, the total time is linear with respect to the input length n .
- **Space Complexity: $O(n)$**
 - In the worst-case scenario—such as an input string of only opening brackets like "(((((" —we push every single character onto the stack.
 - We cannot optimize this to $O(1)$ space because we must store the state of unmatched brackets to verify correct closing order.
 - Thus, space usage grows linearly with the input size.

Alternative Solutions

While the Stack solution is the standard and most efficient approach for interview settings, let's briefly discuss other possibilities and why they fall short.

1. Recursive Parsing:

- One could write a recursive function that checks if the first character matches the last, then recurses on the substring between them.
- *Pros*: Elegant logic.
- *Cons*: Recursion depth limits (Python hits `RecursionError` deep quickly) and overhead make it slower than an iterative stack approach. It also struggles with complex nesting like `( [ { } ] )` unless carefully balanced.

2. Counter with Regex:

- Using regex to match patterns like `\((?:\[^\(\)]|\((?:\[^\(\)]|\(\....\)\)\))*\)`.
- *Pros*: Concise code for specific patterns.
- *Cons*: Regular expressions do not natively handle "counters" or "nested matching" efficiently for all cases (regex engines have limits on recursion depth). It's harder to read and debug than a stack-based algorithm.

The stack approach remains the gold standard because it directly models the problem's constraint: **LIFO ordering**.

Edge Cases

A robust solution must handle scenarios that naive counters often miss.

1. Empty String (`""`):

- The loop runs zero times. The stack remains empty.
- Result: `True`. (Correct, an empty string has no invalid brackets).

2. Single Opening Bracket (`"(", "[", "{"`):

Quick Review

What is the algorithmic pattern for Valid Parentheses?

Use a stack (Last-In-First-Out) to track opening brackets and match them with closing ones.

What is the time complexity of this solution?

$O(n)$ because each character is pushed onto and popped from the stack exactly once.

What is the space complexity for this approach?

$O(n)$ in the worst case where all characters are opening brackets, requiring a stack of size n .

Which edge case causes a simple counter to fail?

Nested or interleaved types like '([)]' pass a simple count but fail structural validation; order matters, not just quantity.

What is a common bug when implementing this with a stack?

Forgetting to check if the stack is empty before popping, or returning true incorrectly if the stack isn't empty at the end.